Intel TDX において IMA 測定値を RTMR へ反映するカーネルモジュールの設計と実装

伊藤 駿[1,2,a)]　今村 拓万[1]　櫻井 碧[1]　林 卓也[1]

概要：Confidential Virtual Machine（CVM）のランタイム測定には Linux Integrity Measurement Architecture（IMA）が用いられる．IMA ログの完全性を検証するため通常は Trusted Platform Module（TPM）の Platform Configuration Register（PCR）を用いるが，Intel TDX 上の CVM では Runtime Measurement Register（RTMR）を用いる方式が提案されている．IMA と RTMR の連携機構は複数提案されているが，様々な理由により現行のカーネルには実装されていない．そこで本稿では，この連携をカーネルモジュールとして実現する方式を提案する．これによりカーネル本体を改変せず，カーネル更新時のパッチ適用に伴う保守負担を軽減できる．PoC として ima-rtmr-extend カーネルモジュールを実装し，Google Compute Engine（GCE）上の CVM において正しく動作することを実証した．

キーワード：Intel TDX，IMA，RTMR，リモートアテステーション，カーネルモジュール

Design and Implementation of a Kernel Module for Extending IMA Runtime Measurements into an RTMR on Intel TDX

Shun Ito[1,2,a)]　Takuma Imamura[1]　Ao Sakurai[1]　Takuya Hayashi[1]

Abstract: The Linux Integrity Measurement Architecture (IMA) is used for runtime measurement of Confidential Virtual Machines (CVMs). IMA normally uses Platform Configuration Registers (PCRs) in a Trusted Platform Module (TPM) to verify the integrity of its measurement log, whereas mechanisms using Runtime Measurement Registers (RTMRs) have been proposed for Intel TDX CVMs. Several mechanisms for integrating IMA with RTMRs have been proposed, but for various reasons they have not been incorporated into the current kernel. We therefore propose implementing this integration as a kernel module. This approach avoids modifying the kernel core and reduces the maintenance burden of reapplying patches after kernel updates. As a proof of concept, we implemented the ima-rtmr-extend kernel module and demonstrated its correct operation on a CVM running on Google Compute Engine (GCE).

Keywords: Intel TDX, IMA, RTMR, Remote Attestation, Linux Kernel Module

1. はじめに

Trusted Execution Environment (TEE) は，ハードウェアが強制する隔離境界内でセキュリティ上重要なソフトウェアを実行する環境である．仮想マシン単位で TEE を構成する形態を Confidential VM （CVM） と呼び，Intel Trust Domain Extensions （TDX），AMD Secure Encrypted Virtualization-Secure Nested Paging （SEV-SNP），Arm Confidential Compute Architecture （CCA） などが存在する [1], [2], [3]．本稿は Intel TDX を対象とする．TDX によって保護された CVM を Trust Domain（TD） と呼ぶ．

リモートアテステーションは，遠隔の実行環境が生成した evidence を検証し，その状態を評価する仕組みであ

[1] 株式会社 Acompany，Confidential Computing Lab
Confidential Computing Lab, Acompany Co., Ltd.
[2] 群馬大学 情報学部
Faculty of Informatics, Gunma University
[a)] shun.ito@acompany-ac.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

る．RFC 9334 の用語に従い，evidence を生成する役割を attester，evidence を評価する役割を verifier，Attestation Result に基づいて判断する役割を relying party と呼ぶ [4]．Linux を動かす CVM では，起動時の初期内容，ファームウェアからカーネルまでのブート過程，および起動後のソフトウェアを段階的に測定できる．これらの測定値を保護された測定レジスタに記録し，署名付きの evidence に含めることで，verifier は実行環境の状態を評価できる．例えば GCE の Confidential VM はこの形のリモートアテステーションを提供している [5]．

Linux Integrity Measurement Architecture（IMA）は，ファイルの実行や読み込みに応じて対象を測定し，計算された測定値を IMA ログへ記録する．Trusted Platform Module（TPM）を持つ構成では，測定値のハッシュチェーンを TPM の Platform Configuration Register（PCR）10 へ記録（extend）する．TPM は，発行時に指定した PCR 値のダイジェストを Attestation Key で署名し，TPM Quote として返す．verifier は IMA ログから PCR 10 の期待値を再計算し，TPM Quote に含まれるダイジェストと照合することで，IMA ログが改ざんされていないことを検証できる [6], [7], [8], [9]．

クラウド VM では，物理 TPM の代わりにホストが提供する vTPM が利用される場合が多い．例えば Google Compute Engine （GCE） の Shielded VM ではホスト上で動作するソフトウェア TPM が CVM に提供される [5], [10]．しかし CVM ではホストを信頼しないため，ホスト上で動作しているソフトウェア TPM も信頼しない．vTPM をホストと通常のゲスト OS の双方から隔離する方式も提案されている [11]．ただし導入にはホスト側の対応が必要であり，クラウド上の CVM に利用者が後から追加することはできない [12], [13]．

Intel TDX では PCR の代替として利用できる RTMR を備えている [2]．TDX においては，RTMR を含むレポートを Intel の Root CA に紐づいた Attestation Key によって署名することで TD Quote を発行し，verifier へ渡す [14]．IMA が測定値を IMA ログへ記録する際に RTMR への extend も実行する機構が実現されれば，verifier は IMA ログから RTMR を再計算し，TD Quote 内の値と照合できる．

しかし，現行の Linux カーネルには，IMA による測定値を RTMR へ extend する機構が実装されていない．Intel の開発者らによる CIMA には，この機構を実装するカーネルパッチが含まれるが，このパッチは Linux メーリングリストへ投稿されておらず，カーネルのメインラインにもマージされていない [15]．IMA Root of Trust Framework でも，この機構の実装が提案されているが，本稿執筆時点でメインラインにはマージされていない [16]．したがって，これらの方式を利用するには，カーネルへのパッチ適用と再コンパイルが必要となる．

そこで本稿は，IMA 本体を変更せず，IMA ログから測定値を取り出し，Linux の `tsm-mr`[17], [18], [19] を介して RTMR へ extend するカーネルモジュールを設計・実装する．カーネルモジュールとして実装することで，カーネル更新時に IMA へのパッチを再適用してカーネルを再コンパイルする保守負担を軽減できる．現時点で `tsm-mr` に対応するゲストドライバは TDX に限られるが，将来 Arm CCA などが対応した場合にも，TEE 固有の命令を本モジュールへ追加せず，同じ経路を利用できる．

本稿の貢献は三点である．第一に，IMA が確定した順序を保って測定値を非同期に RTMR へ反映する本モジュールの設計を与える．第二に，本モジュールを用いて RTMR に extend された IMA ログを検証するための条件と手順を整理する．第三に，GCE 上の Intel TDX 対応 Confidential VM インスタンスで，本モジュールの機能を評価する．

2. IMA と RTMR を用いたリモートアテステーション

2.1 測定から検証までの流れ

IMA は Linux Security Module によるフックを起点としてファイルを測定する機構である．

IMA はファイルパスやコンテンツから計算される測定値を template digest として追記専用の IMA ログへ記録するとともにハッシュチェーンを持つ測定レジスタに extend する．attester は測定レジスタの状態を evidence に含めて署名する．verifier は evidence の署名を検証し，IMA ログの template digest を記録順に extend して測定レジスタの期待値を再計算する．その値が evidence の示す状態と一致すれば，測定レジスタへ反映された範囲について，IMA ログのエントリが測定後に変更，削除，追加または並べ替えられていないことを確認できる [6]．

TPM を利用する構成では，IMA の測定レジスタとして通常 PCR 10 を用いる．TPM における evidence には TPM Quote が用いられる．TPM Quote は選択された PCR 値のダイジェストを含み，TPM 内で保護された Attestation Key （AK） によって署名される．verifier は TPM ベンダによる Root CA を用いて AK Public key を検証した後に AK Public Key を用いて TPM Quote を検証する．その後 IMA ログから再計算した PCR 10 と Quote 内のダイジェストとの整合性を確認する [7], [8], [9]．

IMA と Intel TDX を連携する構成では，測定レジスタとして RTMR を用いる．TDX における evidence には TD Quote が用いられる．TD Quote は RTMR 値を含み，TD Quoting Enclave 内の AK によって署名される．verifier は Intel による Root CA を用いて AK Cert までの証明書チェーンを検証し，AK Public Key を用いて TD Quote を検証する．それ以降は TPM と同様に RTMR と

© 2026 Information Processing Society of Japan

TD Quote 内の RTMR との整合性を確認する [14].

2.2 TDX の測定レジスタと tsm-mr

Intel TDX は，TD の初期内容を表す測定値を格納する MRTD とは別に，実行時に extend できる RTMR 0 から RTMR 3 までの 4 つの RTMR を備える．各 RTMR は，SHA-384 ダイジェストの extend によって更新されるハッシュチェーンを保持する．現在値 R_n と 48 バイトの SHA-384 ダイジェスト h_n に対する extend は，次式で定義される [2].

$$R_{n+1} = \text{SHA-384}(R_n \| h_n) \tag{1}$$

TD Virtual Firmware （TDVF） やブートローダは，ブート中の測定値を RTMR へ extend する．対応するイベントは CC event log（CCEL）に記録され，OS へ渡される [20], [21]．RTMR へ extend する値は，TDVF やクラウド環境の仕様によって異なる．Intel の TDVF Design Guide は，RTMR 2 を OS カーネル，initrd およびアプリケーションなど，従来の PCR 8–15 に対応する測定へ割り当てている [21]．IMA が通常用いる PCR 10 もこの範囲に含まれる [6], [8]．一方，GCE では，RTMR 2 にカーネルとコマンドライン，RTMR 3 にユーザーランドから渡される追加イベントログを記録する [22].

Linux 6.16 では，TEE Security Manager（TSM）[23] の `tsm-mr` と，TDX の RTMR を `tsm-mr` から利用可能にする TDX ゲストドライバの機能がカーネルのメインラインに導入された [18], [19], [24]．本稿が用いる `/sys/class/misc/tdx_guest/measurements/rtmr2:sha384` は TDX ゲストドライバによって sysfs に公開されているインターフェースであり，RTMR 2 の現在値を読み出せるほか，48 バイトの値を書き込むと，その値が RTMR 2 へ extend される [17], [19].

2.3 既存方式と未解決の問題

CVM で vTPM（software TPM）を IMA の信頼の起点として利用するには，vTPM をホスト VMM と通常のゲスト OS の双方から隔離し，vTPM を実装するコードと Attestation Key （AK） を CVM のハードウェアレポートに結び付ける必要がある．verifier は，IMA ログから再計算した PCR 10 と vTPM Quote の整合性，および vTPM の AK による Quote の署名を検証する．さらに，その AK と vTPM を実装するコードが CVM のハードウェアレポートに結び付いていることを確認することで，IMA ログの検証をハードウェアの Root of Trust まで連鎖させる [11].

OpenHCL はこのアーキテクチャの vTPM を実現する実装の 1 つである．Intel TDX の TD Partitioning を用い，ホスト VMM から保護された TD 内の Virtual Trust Level 2 （VTL2）で vTPM を動作させる．通常のゲスト OS は VTL0 で動作し，VTL2 のメモリや資源へアクセスできない [12], [13].

ただし，OpenHCL はホスト VMM が VM の起動時に Independent Guest Virtual Machine （IGVM） イメージをロードする必要がある．OpenHCL の公式ガイドによれば，2026 年 7 月 30 日時点で IGVM イメージをロードできる VMM は Hyper-V と OpenVMM であり，KVM には必要な機能が実装されていない [25], [26]．そのため，現時点で KVM を用いる VMM では OpenHCL を利用できない．

また，この IGVM のロードを行うにはホスト VMM の操作が必要であり，クラウド利用者が既存の TDX ゲストへ後から導入することはできない．GCE などのクラウド上の CVM で利用するにはクラウド事業者側の対応が必要となり，利用者が自ら導入する場合は，ホストを制御できるベアメタル環境が必要である．

Intel の `vtpm-td` は，vTPM を同一ホスト内でゲストとは独立した TD として実装することで，VMM を TCB に含めずに vTPM 機能を提供する方式である．しかし，Intel による開発は終了し，リポジトリもアーカイブされている [27].

TDX CVM では，2.1 節で述べた，vTPM を用いずに IMA ログを RTMR と TD Quote によって検証する方式を適用できる．しかし，IMA ログに記録した測定値を RTMR へ extend する機能はカーネルのメインラインに存在しない．この機能を追加するカーネルへの実装として，幾つかの手法が提案されている．

cc-api の CIMA は，IMA 本体と TDX 側へパッチを適用し，IMA ログへの追記に続けて TDX 固有の RTMR extend を呼び出す実装の 1 つである [15]．また，IMA Root of Trust Framework は，TPM 以外を含む複数の Root of Trust を IMA から扱う抽象化を提案する [16].

しかしながら，2026 年 7 月 29 日時点でこれらのパッチは Linux カーネルのメインラインにはマージされていない．カーネルメーリングリストでは，ログ記録とレジスタ更新の原子性 [28]，複数の Root of Trust でも既存の IMA 保証を維持できるか [29]，アーキテクチャ依存処理を配置する層 [30] が指摘された後，議論は停滞しており，マージの見通しは立っていない．

3. 提案方式：ima-rtmr-extend

本稿では，IMA 本体を変更せず，IMA ログに記録された測定値を RTMR 2 へ反映するカーネルモジュール ima-rtmr-extend（以下，本モジュール）を設計する．本モジュールは，IMA ログへの追記完了を検出し，記録された template digest を `tsm-mr` が公開する sysfs へ書き込むことで RTMR 2 へ extend する．図 1 に，IMA による測

© 2026 Information Processing Society of Japan

定から，本モジュールが測定値を RTMR 2 へ extend し，TD Quote に反映されるまでの処理を示す．本節では，対象環境と脅威モデル，IMA ログへの追記の検出，RTMR を extend する方法，およびモジュールロード前の RTMR 値の扱いを説明する．

3.1 対象環境と脅威モデル

本稿は，TD 上で動作するユーザーランドアプリケーションをリモートから利用する relying party を想定する．そのため，通常の CVM の脅威モデルとは異なりゲストユーザーランドを TCB の範囲外とする．

攻撃者はホスト OS，ハイパーバイザ（VMM）およびクラウド制御プレーンを制御できるものとする．プラットフォーム側の TCB には CPU，マイクロコード，プラットフォームファームウェア，TDX Module および TDQE を含める．TD 内の TCB には TDVF，ゲストカーネル，IMA，`tsm-mr`，TDX ゲストドライバ，本モジュールおよび attester の処理を含める．通常のゲストユーザーランドは TCB に含めず，ゲストカーネル権限を取得した攻撃者は対象外とする．

verifier は Intel のルート証明書を trust anchor とし，PCK 証明書チェーン，失効情報，TCB Info，TDQE Identity および TD Quote の署名を検証する．さらに，MRTD, TDATTRIBUTES, XFAM などを Reference Values と appraisal policy に照らして評価した後，本方式固有の IMA ログ検証を行う．

本稿は，TD Quote 署名の偽造困難性および SHA-384 の衝突耐性を仮定する．第三者による対象 RTMR への extend は IMA ログとの不一致を生じさせるため，これに起因する可用性攻撃は対象外とする．物理攻撃，サイドチャネル攻撃およびフォールトインジェクションも対象外とする．

3.2 IMA ログへの追記の検出

本モジュールではカーネルランド内で動作する IMA がログを記録する関数の直後にフックを仕掛けることで RTMR へ extend させる処理を追加で行わせる．図 1 の破線で示すように，IMA が測定値を追加するカーネル内の関数 `ima_add_template_entry` はログの記録を行う関数である．この関数に対して，return 時に任意の関数を probe できる kretprobe を用いることでカーネルモジュールから処理を挿入する [31]．しかし，kretprobe の return handler は atomic context で動作するため，sleep する可能性がある sysfs への `kernel_write` による RTMR の extend を直接実行できない．そこで，return handler はログの取得と extend をワーカに委託する．ワーカは workqueue に登録され，IMA ログを記録順に走査し，その順序で RTMR を extend する．具体的なログ走査，シンボル解決およびカーソル管理は付録 A.2 に示す．

3.3 `tsm-mr` による RTMR の extend

Linux カーネルランドにおいて RTMR を extend する方法は 2 つ存在する．1 つ目は TDX Module に対して `TDG.MR.RTMR.EXTEND` を実行することで直に extend を行う方法である．2 つ目が Linux カーネルが sysfs 上に公開する tsm-mr インターフェース `/sys/class/misc/tdx_guest/measurements/rtmr2:sha384` を用いて extend する方法である．

本モジュールは，図 1 の実線で示すように，RTMR を extend する TDX 固有の TDCALL `TDG.MR.RTMR.EXTEND` を直接呼び出さず，RTMR の extend を `tsm-mr` と委ねる．現時点で `tsm-mr` に対応するゲストドライバは TDX に限られるが，将来ほかの TEE が対応した場合にも，`tsm-mr` を介して測定レジスタを extend する同じ方法を利用できる．

IMA ログの extend 先として RTMR 0 から RTMR 3 のうち RTMR 2 をデフォルトで用いる．TDVF Design Guide では，RTMR 2 を OS カーネル，initrd およびアプリケーションなどの OS コンポーネントに割り当てているためである [21]．

extend の際には，SHA-384 ベースの template digest を IMA ログから選び extend する．ただし，正常な測定ができなかった際に記録される violation エントリでは，全 `0xFF` として extend する [6]．ワーカは，未処理のエントリを取得し，IMA が確定した記録順のまま RTMR に extend する．ワーカは並列実行されることはないため，同一エントリが複数回 extend されることはない．

`tsm-mr` が公開する sysfs への書き込みは失敗する場合がある．このとき，そのまま後続の extend を行ってしまうと IMA ログからエントリを一部欠落させた状態で RTMR に extend されてしまう．その結果，以降の RTMR の検証に常に失敗してしまい，これはゲストが再起動されるまで回復しない．これを防ぐため，失敗した場合は後続の書込みを停止し，失敗前までに反映した IMA ログとの対応を維持する．このため，書込みの成功を確認した場合にだけ次のエントリの処理に進む．停止したかどうかは `/sys/kernel/ima_rtmr/disabled` で確認できる．

3.4 モジュールロード前の測定値の扱い

カーネルが起動する前に TDVF やブートローダは測定ログである CCEL を RTMR 2 に extend している．その後，カーネルは本モジュールがロードされるより前に IMA ログへ記録を開始する．いくつかのログが記録された後に本カーネルモジュールが読み込まれ，RTMR への extend が開始される．

本モジュールは CCEL 等が既に extend されている

© 2026 Information Processing Society of Japan

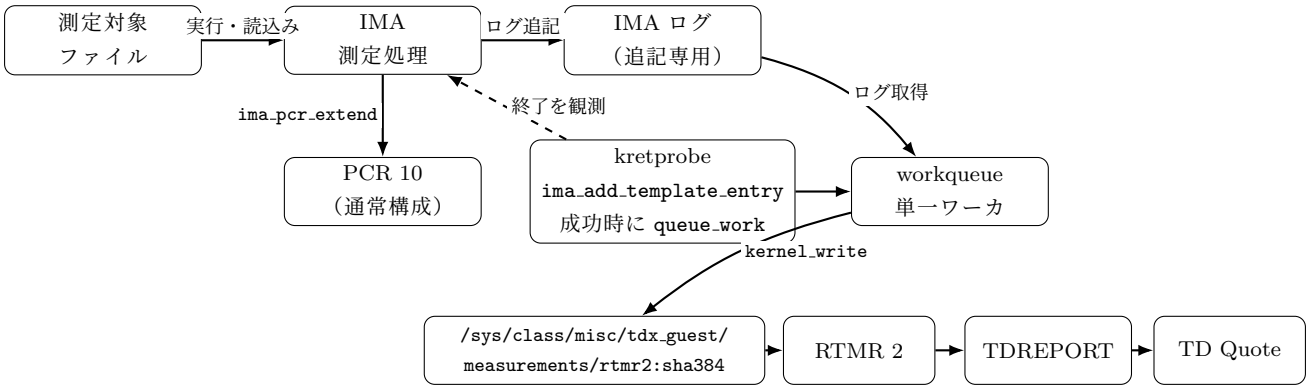


図 1 IMA ログの測定値を RTMR 2 へ extend し，TD Quote に反映されるまでの処理．実線は測定値と処理の流れ，破線は kretprobe による関数終了の観測を表す．

Fig. 1 Processing flow for extending IMA measurements into RTMR 2 and including them in a TD Quote.

RTMR 2 に続けて extend を行う．モジュールのロード直後の RTMR 2 を `initial` として保持し，`/sys/kernel/ima_rtmr/initial` で公開する．

カーネルツリー外（out-of-tree; OOT）の本モジュールをロードした時点では，IMA ログに既存のエントリが存在し得る．モジュールロード前に記録される IMA ログの詳細を付録 A.1 に示す．ロード前に記録されていた既存のログ件数を `skip_count` として保存し，以後に追加されたログだけを処理する．`skip_count` は `/sys/kernel/ima_rtmr/skip_count` で公開する．

4. リモートアテステーションの手順

4.1 検証条件

IMA ログを $L = (e_1, e_2, \ldots)$ とする．各エントリ e_i について RTMR へ extend する 48 バイトの値を $h(e_i)$ とする．e_i が violation エントリでない場合，$h(e_i)$ は SHA-384 の template digest であり，violation エントリである場合は全 `0xFF`（48 バイト）である．また，各エントリ e_i に対応する時点での RTMR 2 の値を R_i とする．本モジュールがロードされた時点での IMA ログに記録済みのエントリの数を s とする．この s は `skip_count` と一致する．この時点で L には e_s までの IMA ログが記録されており，RTMR 2の値は R_s である．この R_s は `initial` と一致する．

モジュールのロード以降に IMA ログに追記されたエントリ e_t は，RTMR 2 に extend されるため以下が成立する．

$$R_t = \text{SHA-384}\left(R_{t-1} \| h(e_t)\right) \quad \text{for } t > s. \tag{2}$$

4.2 attester の手順

attester はまず TD Quote Q を取得する．Q の中の RTMR 2を R_q とする．この Q は e_q が IMA に記録された後かつ e_{q+1} が記録される前に取得されたものである．この操作は本モジュールのロード後に行われるので $q > s$ が成り立つ．

次に attester は IMA ログのスナップショット $L' = (e_1, e_2, \ldots, e_t)$ を取得する．これは TD Quote よりも後に取得されなければならない．従って $t \geq q$ が成り立つ．

その後，sysfs より取得した s（`skip_count`）と R_s（`initial`）と共に TD Quote Q と IMA ログ L' を verifier に送信する．

4.3 verifier の手順

verifier は attester より受け取った L'，s，R_s をもとに R_{s+1}，R_{s+2}，... を計算する．この計算は漸化式 2 に従って行う．

計算した RTMR 2 の値の列に受け取った Q 内の R_q が含まれていれば，e_{s+1} から e_q までの IMA ログは改ざんされていないことを検証できたといえる．もし含まれていなかったとすると検証に失敗したということになる．

もし $t \geq q$ が成立しない，すなわち attester が TD Quote の取得よりも前に IMA ログのスナップショットを取得した場合，R_{s+1}，R_{s+2}，...，R_t の中に R_q が含まれないため常に検証に失敗してしまう．そのため，attester は常に IMA ログのスナップショットを TD Quote の後に取得しなければならない．

5. 本実装の限界

5.1 カーネルモジュールの限界

本実装には，カーネルモジュールという形態を採用したことによる幾つかの限界が存在する．

3.4 節で述べたとおり，本モジュールがロードされる以前から IMA ログには測定値が記録されているが，現実装はそれらを処理せず，ロード後に追加された測定値だけを RTMR 2 へ extend する．従って，ロード前の IMA ログは 4.3 節の手順では検証できない．また，本モジュールがアンロードされている間に追加された測定値も RTMR 2

© 2026 Information Processing Society of Japan

へ extend されない．カーネルモジュールはアンロード中に状態を保持できないため，再ロード時には IMA ログのどのエントリまでが RTMR 2 へ extend 済みであるかを判別できない．

これらの測定値を後から RTMR 2 へ extend する機能を実装すれば，未処理の IMA ログを反映できる．しかし，現実装では未処理の範囲を特定できないため，この機能を実装していない．一方，本モジュールの機能をコンパイル時にカーネルへ組み込めば，IMA が測定を開始する前から動作させられるうえ，アンロードも起こらない．これにより，ロード前およびアンロード中の測定値という問題が解決される．ただし，この方法ではカーネルの更新ごとに再コンパイルが必要となる．

5.2 実装上の制約

本モジュールは IMA 本体の内部関数にフックする機構である．従って，この内部関数に大きな変更が加えられるようなカーネルの改変があった際には本モジュールは動作しなくなる．カーネルの通常のアップデートであれば影響はないが，IMA そのものが大幅に変更されるようなアップデートの際には本モジュールの修正が必要になる．

また，本モジュールでフックしている関数や構造体は export されていないため，これらのシンボルがコンパイル時最適化によって喪失してしまう可能性がある．その場合にも本モジュールの修正が必要になる．

kretprobe の制約として kretprobe instance 数の上限が存在する．これは，probe している関数が短時間に大量に呼び出されると一定数以上はフックができなくなるというものである．この回数は `nmissed` に記録される [31]．そのため，高負荷時には extend が極端に遅れる場合がある．この上限によって extend されなかった IMA ログは次のフック時にまとめて処理される．すなわち次のフックが行われるまでの間は extend されない．この制約については 6.2 節で検証している．

6. 評価

本評価の目的は次の三点である．第一に，IMA ログへ追加された測定値が記録順に一度ずつ RTMR 2 へ extend され，反映が一時的に遅れても，測定対象の実行終了後にはログ末尾までの処理が完了することを確認する．第二に，本モジュールが測定対象プログラムの実行時間へ与える影響を測る．第三に，sysfs への書込みに失敗した場合に，後続の extend が停止されることを確認する．

6.1 実験条件

実験には GCE 上の Intel TDX 対応 Confidential VM インスタンス（`c3-standard-44`，44 vCPU）を用いた．OS は Ubuntu 25.10，カーネルは `6.17.0-1013-gcp` である．IMA は SHA-384 と `ima-ng` template を用い，専用の ext4 ファイルシステム上で実行したプログラムだけを測定した．Secure Boot は無効とし，評価中は本モジュール以外の主体が対象 RTMR へ extend しないものと仮定する．IMA ポリシーと検証手順の詳細は付録 A.3 に示す．

表 1 並列度を変えた測定値の書込み結果
Table 1 Results with different workload parallelism.

並列度	対象ログ件数	書込み件数	`nmissed`	再計算値
1	4097	4097	0	一致
44	4097	4097	0	一致
176	4097	4097	2050	一致

表 2 処理中の未処理件数と extend 完了までの遅延時間
Table 2 Pending measurements and detected completion time.

並列度	最大未処理件数	遅延時間 [ms]	`nmissed`	再計算値
44	1	5.1	0	一致
176	21	4.9	2004	一致

6.2 RTMR への extend の確認

本評価では本カーネルモジュールを用いた IMA ログの RTMR への extend が正常に行われることを確認する．そのために 4096 個の実行ファイルと，実験対象の末尾を示す 1 個の確認用実行ファイルを実行し，それらを IMA に測定させた．成功条件は，全てのエントリが正しく extend され，ログ末尾まで再計算したハッシュチェーンが RTMR 2 と一致することである．

本モジュールを条件ごとに再ロードし，並列度 1，44，176 について各 1 回，4096 個のファイルと末尾確認用ファイルを実行した．結果を表 1 に示す．各条件で IMA ログの記録順にログ末尾まで再計算した値が RTMR 2 と一致した．

並列度 176 では `nmissed` が 2050 回発生したが，この試行では後続のワーカが未処理分を取得し，末尾確認用ファイルを含む全件を処理できた．ただし，最後のエントリ追加時に関数の終了を検出できず，その後にログが追加されない場合には，未処理分が残る可能性がある．この制約は 5.2 節で述べた．

非同期処理中には，IMA ログへの追加に RTMR への反映が追いつかず，未処理の測定値が一時的に生じる可能性がある．そこで，未処理件数が生じうる並列化したケースについて，表 1 とは別に各 1 回の試行を行い，成功した IMA ログへの追加件数と，RTMR への反映が完了した件数をイベントごとに記録し，両者の差を未処理件数とした．結果を表 2 に示す．いずれの並列度でも一時的な未処理が発生した．いずれも最後には未処理件数が 0 となり，ログ末尾までの再計算値が RTMR 2 と一致した．

© 2026 Information Processing Society of Japan

表 3　6 ブートにおける実行時間の比較
Table 3　Paired timing comparison over six boots.

指標	結果
B0 実行時間の中央値	8013.6 ms
B1 実行時間の中央値	8083.3 ms
各ブート内の実行時間差の中央値	35.2 ms
実行時間差の 95%信頼区間	9.9–86.6 ms
各ブート内の実行時間比の中央値	1.0044
実行時間比の 95%信頼区間	1.0012–1.0108
B1 書込み完了検出時間の中央値	4.9 ms

6.3　測定対象プログラムの実行時間に与える影響

本評価では，RTMR の extend が測定対象プログラムの実行時間に与える影響を評価する．

IMA ポリシーだけを有効にした条件を B0，本モジュールも有効にした条件を B1 とし，4096 個の測定対象を並列度 44 で実行した．異なる boot ID を持つ 6 回のブートをそれぞれ一組とし，各ブート内で B0 と B1 を一回ずつ実行した．実行順による偏りを抑えるため，奇数回目のブートでは B0 の後に B1，偶数回目では B1 の後に B0 を実行した．各試行では新しい測定対象ファイルを生成し，最初のプログラムの実行開始から 4096 個すべての終了までを実行時間とした．

表 3 に結果を示す．本モジュールを有効にした B1 の実行時間は B0 より長かったが，その増加は実行時間全体に対して 1%程度であり，本環境における影響は小さかった．

6.4　sysfs への書込み失敗時の停止

3.3 節で述べたように，sysfs への書き込みが失敗した際には，それ以降の extend を中止し動作を停止する．本評価ではこの機構が正しく動作することを検証した．

評価用に，任意のタイミングで sysfs に対する書き込みエラーを発生させるカーネルモジュールを作成した．一回の試行で 32 個の測定対象と末尾確認用ファイルを逐次実行し，10 回目の書込みを失敗させたところ，`disabled=1` となり，RTMR 2 は先頭 9 件を再計算した値と一致した．したがって，本モジュールが検出できる書込みエラーが発生した場合に，失敗したエントリ以後を反映せず，モジュールロード期間内で処理を停止することを確認した．

7.　結論

本稿は，IMA ログの template digest を `tsm-mr` を介して RTMR 2 へ extend するカーネルモジュールを設計・実装した．GCE 上の Intel TDX 対応 Confidential VM インスタンスにおける評価では，IMA ログの記録順でログ末尾までの測定値を RTMR 2 へ反映し，再計算値が RTMR 2 と一致すること，本環境では測定対象プログラムの実行時間への影響が小さいこと，および sysfs への書込み失敗後に後続処理を停止することを確認した．

本方式は IMA 本体を変更せず，カーネルのメインラインに導入された `tsm-mr` を利用するため，カーネル更新時に IMA へのパッチを再適用してカーネルを再コンパイルする保守負担を軽減できる．

一方，カーネルモジュールという設計に起因する測定の限界が存在した．これはカーネルに組み込むことで解決できるが，その場合にはカーネルのアップデートの度に再コンパイルが必要となり運用コストが大きい．カーネルモジュールと同等程度の運用コストを維持しつつ測定限界を解決する手法の模索については今後の課題となる．

参考文献

[1] M. Misono, D. Stavrakakis, N. Santos, and P. Bhatotia: Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX, *Proc. ACM on Measurement and Analysis of Computing Systems (POMACS)*, Vol. 8, No. 3, Article 36, pp. 1–42 (2024), doi:10.1145/3700418.

[2] Intel Corporation: *Intel TDX Module v1.5 Base Architecture Specification*, Document 348549-002US (2023), `https://cdrdv2-public.intel.com/733575/intel-tdx-module-1.5-base-spec-348549002.pdf`.

[3] Arm Limited: *Learn the architecture — Realm Management Extension guide*, Document ID DEN0126, Version 1.1, Issue 02 (2025), `https://documentation-service.arm.com/static/67aca7b96dbc975ccea92d2f`.

[4] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan: Remote ATtestation procedureS (RATS) Architecture, RFC 9334 (2023), `https://www.rfc-editor.org/rfc/rfc9334.html`.

[5] Google Cloud: Remote attestation overview, Confidential VM Documentation, `https://docs.cloud.google.com/confidential-computing/confidential-vm/docs/attestation-overview` (accessed 2026-07-30).

[6] IMA contributors: IMA Event Log, IMA 1.0 Documentation, `https://ima-doc.readthedocs.io/en/latest/event-log-format.html` (accessed 2026-07-23).

[7] Trusted Computing Group: *Trusted Platform Module Library, Part 1: Architecture*, Family 2.0, Level 00, Revision 01.83 (2024), `https://trustedcomputinggroup.org/wp-content/uploads/TPM-2.0-1.83-Part-1-Architecture.pdf`.

[8] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn: Design and Implementation of a TCG-based Integrity Measurement Architecture, *Proc. 13th USENIX Security Symposium*, pp. 223–238 (2004).

[9] N. Schear, P. T. Cable II, T. M. Moyer, B. Richard, and R. Rudd: Bootstrapping and Maintaining Trust in the Cloud, *Proc. 32nd Annual Computer Security Applications Conf. (ACSAC '16)*, pp. 65–77 (2016), doi:10.1145/2991079.2991104.

[10] M. Zimmerman: Virtual Trusted Platform Module for Shielded VMs: security in plaintext, Google Cloud Blog (2018), `https://cloud.google.com/blog/products/identity-security/virtual-trusted-platform-module-for-shielded-vms-security-in-plaintext`.

[11] V. Narayanan, C. Carvalho, A. Ruocco, G. Almási, J. Bottomley, M. Ye, T. Feldman-Fitzthum, D. Buono, H. Franke, and A. Burtsev: Remote Attestation of Con-

© 2026 Information Processing Society of Japan

fidential VMs Using Ephemeral vTPMs, *Proc. 39th Annual Computer Security Applications Conf. (ACSAC '23)*, pp. 732–743 (2023), doi:10.1145/3627106.3627112.
[12] Microsoft: OpenHCL Architecture, *The OpenVMM Guide*, https://openvmm.dev/guide/reference/architecture/openhcl.html (accessed 2026-07-30).
[13] Intel Corporation: *Intel TDX Module TD Partitioning Architecture Specification*, Document 354807-006US (2026), https://cdrdv2-public.intel.com/865799/intel-tdx-module-td-partitioning-spec-354807006.pdf.
[14] Intel Corporation: *Intel TDX Data Center Attestation Primitives: Quote Generation Library and Quote Verification Library*, Rev. 0.9 (2026), https://download.01.org/intel-sgx/sgx-dcap/1.27/linux/docs/Intel_TDX_DCAP_Quoting_Library_API.pdf (accessed 2026-07-30).
[15] cc-api contributors: CIMA kernel patches for extending IMA measurements to TDX RTMR, Container Integrity Measurement Agent, commit f0951a041b9e680c640b28e46ef02a45d55c98b8 (2025), https://github.com/cc-api/container-integrity-measurement-agent/tree/f0951a041b9e680c640b28e46ef02a45d55c98b8/tools/build/kernel/patches.
[16] R. Gong: [PATCH -next RFC 0/4] IMA Root of Trust (RoT) Framework, linux-integrity mailing list (2025), https://www.mail-archive.com/linux-integrity@vger.kernel.org/msg04003.html.
[17] Linux kernel community: Measurement Registers, Linux Kernel Documentation, https://www.kernel.org/doc/html/v6.16/driver-api/coco/measurement-registers.html (accessed 2026-07-22).
[18] C. Xing: tsm-mr: Add TVM Measurement Register support, Linux kernel commit b9e22b35d4598aefed642928ed2856a9900e5b37 (2025), https://github.com/torvalds/linux/commit/b9e22b35d4598aefed642928ed2856a9900e5b37.
[19] C. Xing: virt: tdx-guest: Expose TDX MRs as sysfs attributes, Linux kernel commit 4d2a7bfad5b79b3a362fc7cfed8d7ddbd263a838 (2025), https://github.com/torvalds/linux/commit/4d2a7bfad5b79b3a362fc7cfed8d7ddbd263a838.
[20] UEFI Forum: *Advanced Configuration and Power Interface (ACPI) Specification, Release 6.5*, Sec. 5.2.34 (2022), https://uefi.org/specs/ACPI/6.5/05_ACPI_Software_Programming_Model.html#cc-event-log-acpi-table.
[21] Intel Corporation: *Intel TDX Virtual Firmware Design Guide*, Rev. 004, Document 344991-004US (2023), https://cdrdv2-public.intel.com/733585/tdx-virtual-firmware-design-guide-rev-004-20231206.pdf.
[22] Google Cloud: Confidential VM measurement register contents, Confidential VM Documentation, https://docs.cloud.google.com/confidential-computing/confidential-vm/docs/measurement-register-contents (accessed 2026-07-22).
[23] D. Williams: coco/tsm: Introduce a core device for TEE Security Managers, linux-pci mailing list, PATCH v4 01/10 (2025), https://lore.kernel.org/linux-pci/20250717183358.1332417-2-dan.j.williams@intel.com/.
[24] C. Xing: tsm-mr: Unified Measurement Register ABI for TVMs, linux-kernel mailing list, v6 (2025), https://patchew.org/linux/20250506-tdx-rtmr-v6-0-ac6ff5e9d58a@intel.com/.
[25] Microsoft: Running OpenHCL, *The OpenVMM Guide*, https://openvmm.dev/guide/user_guide/openhcl/run.html (accessed 2026-07-30).
[26] Microsoft: Running OpenHCL on Linux, *The OpenVMM Guide*, https://openvmm.dev/guide/user_guide/openhcl/run/openvmm_linux.html (accessed 2026-07-30).
[27] Intel Corporation: rust-vtpm-td, commit 79ef9a3d3eb6a4956611186d3df57844cc0ac67b (2025), https://github.com/intel/vtpm-td/tree/79ef9a3d3eb6a4956611186d3df57844cc0ac67b (archived; accessed 2026-08-20).
[28] J. Bottomley: Re: [RFC PATCH v2 0/4] tsm: Runtime measurement registers ABI, linux-integrity mailing list, msg00941 (2024), https://www.mail-archive.com/linux-integrity@vger.kernel.org/msg00941.html.
[29] M. Zohar: Re: [PATCH -next RFC 0/4] IMA Root of Trust (RoT) Framework, linux-integrity mailing list, msg04059 (2025), https://www.mail-archive.com/linux-integrity@vger.kernel.org/msg04059.html.
[30] S. Kuppuswamy: Re: [RFC PATCH v2 3/4] tsm: Map RTMRs to TCG TPM PCRs, linux-integrity mailing list, msg00878 (2024), https://www.mail-archive.com/linux-integrity@vger.kernel.org/msg00878.html.
[31] Linux kernel community: Kernel Probes (Kprobes), Linux Kernel Documentation, https://www.kernel.org/doc/html/v6.16/trace/kprobes.html (accessed 2026-07-30).
[32] Linux kernel community: ima_add_boot_aggregate, Linux kernel source code, Linux 6.16, commit 038d61fd642278bab63ee8ef722c50d10ab01e8f (2025), https://github.com/torvalds/linux/blob/038d61fd642278bab63ee8ef722c50d10ab01e8f/security/integrity/ima/ima_init.c#L143-L147.
[33] Linux kernel community: ima_calc_boot_aggregate_tfm, Linux kernel source code, Linux 6.16, commit 038d61fd642278bab63ee8ef722c50d10ab01e8f (2025), https://github.com/torvalds/linux/blob/038d61fd642278bab63ee8ef722c50d10ab01e8f/security/integrity/ima/ima_crypto.c#L795-L837.
[34] Linux kernel community: Workqueue, Linux Kernel Documentation, https://www.kernel.org/doc/html/v6.16/core-api/workqueue.html (accessed 2026-07-30).

付　　録

A.1　モジュールロード前の IMA ログ

本モジュールをロードする前にも，複数の要因によって IMA の測定イベントが生成される．boot_aggregate はポリシーの適用より前に記録され [32]，TPM を利用する構成では，使用するハッシュバンクに応じて PCR 0–7 または PCR 0–9 を集約した値となる [33]．TPM のない CVM では，ima_add_boot_aggregate が ima_tpm_chip の不在を検出すると ima_calc_boot_aggregate を呼ばず，全 0 で初期化したダイジェストから template digest を計算す

© 2026 Information Processing Society of Japan

るため，値は一定となる．

このほか，設定と実行状態に応じて，critical_data ポリシーによる kernel_info と kernel_version，kexec 連鎖から復元されるエントリ，SELinux の状態とポリシー，dm-ima，.ima と .ima_blacklist の鍵，IMA ポリシー，kexec のコマンドラインが記録される．測定違反が生じた場合には，ポリシーにかかわらず violation エントリも追加される．

ima_policy=tcb を用い，SELinux や dm-verity などを使用せず，測定違反も生じない最小構成では，モジュールロード前のエントリは boot_aggregate の 1 件となる．critical_data，SELinux，dm-verity，.ima 鍵投入などを併用すると，先行エントリは増える．

A.2 実装機構の詳細

IMA が測定値を追加する関数 ima_add_template_entry の終了を kretprobe で観測し，成功（return 0）のたびに workqueue へ処理を登録する．return handler は登録だけを行い，sleep する可能性がある書込みは単一のワーカへ委譲する．ワーカは前回処理した位置をカーソルとして保持し，実行時に ima_measurements の末尾を取得して，カーソルからその末尾まで順に辿る．

workqueue はワーカの実行開始前に pending 状態を解除するため，ワーカの実行中に queue_work が呼び出されると，次の実行が予約される．その間に queue_work が再び呼び出されても新たな実行は追加されないが，次の実行では IMA ログの末尾を改めて取得するため，その間に追加されたエントリも処理される．同一 work item は同時に実行されず，ワーカは 1 つずつ順に実行される [34]．このため，処理された範囲では IMA が mutex の下で確定した追記順が extend 順になる．

kretprobe でフックした ima_add_template_entry は export されていないため，シンボル名を指定して kretprobe を登録する．ima_measurements と，digest 配列サイズの計算に必要となる ima_extra_slots も export されていない．これらの解決には，kprobe で取得した kallsyms_lookup_name のアドレスを用いる．

ワーカは，IMA ログへの追記と並行して変化する ima_measurements の末尾を安全に取得するため，Read-Copy-Update （RCU） の読み取り側区間に入る．kernel_write は sleep する可能性があり，この区間内では実行できないため，末尾を保存した時点で区間を抜ける．IMA の各エントリは解放されず，削除 API もないため，区間を抜けた後も保存したカーソルから末尾までを走査し，sysfs へ書き込める．

tsm-mr が公開する sysfs への kernel_write がエラーを返すか，RTMR が用いる SHA-384 に対応するダイジェストが IMA ログのエントリにない場合は，disabled を設定して以後の extend を停止し，不完全な状態で RTMR への extend が行われることはない．

A.3 評価条件の詳細

カーネルコマンドラインで ima_hash=sha384 と ima_template=ima-ng を指定し，専用の ext4 ファイルシステム上で実行したプログラムだけを測定する次の IMA ポリシーを設定した．

```
measure func=BPRM_CHECK
fsuuid=54d785b0-0662-4223-b39b-736793576a05
pcr=10 template=ima-ng
```

このポリシーにより，実験スクリプトや状態確認コマンドによる測定値の追加を避け，測定対象プログラムの終了後は新しい測定値が追加されないようにした．

各試行では，処理内容を変えずに異なる測定値を得るため，/bin/true を別々の inode へ複製し，ファイル末尾へ試行番号と連番を付けて，内容の異なる 4096 個のファイルを生成した．最後に，実験対象の末尾を明示するため，内容と inode がほかの測定対象と異なる末尾確認用ファイルを同じ方法で生成して実行した．したがって，モジュール有効時に追加される測定値は合計 4097 件である．対応表にはパス，inode およびファイルの SHA-384 ダイジェストを記録し，IMA ログとの一対一対応と，末尾確認用ファイルが実験対象の最後にあることを確認した．予備実験では，同じ inode の再実行では測定値が増えず，内容と inode の異なるファイルでは一件ずつ増えることも確認した．

検証プログラム validate.py には，initial と skip_count で示される位置からのみ再計算し，開始位置を探索しない条件を追加した．測定対象の実行後は，各回の読出しの間に 1 ms の sleep を挟みながら extended_count を読み，対象件数を初めて観測した時点を完了検出時点とした．さらに，ログ件数と状態値が少なくとも 3 回，1 秒間隔の読出しで変化しないことを確認してから，IMA ログと RTMR 2 を取得した．測定対象バイナリ，モジュール，カーネルコマンドライン，IMA ポリシーおよび実験結果のハッシュ値を保存した．

A.4 実装の公開

本稿で実装した ima-rtmr-extend カーネルモジュールと検証プログラムを次のリポジトリで公開している．https://github.com/acompany-develop/ima-rtmr-extend

© 2026 Information Processing Society of Japan